HEITOR RAMOS VIEIRA

GABRIEL COELHO COSTA

OTÁVIO SBAMPATO ANDRADE

RELATÓRIO TRABALHO PRÁTICO

COMPILADORES:

ETAPA 1

1ª edição

LAVRAS – MG

2026

HEITOR RAMOS VIEIRA
GABRIEL COELHO COSTA
OTÁVIO SBAMPATO ANDRADE

RELATÓRIO TRABALHO PRÁTICO COMPILADORES:

ETAPA 1

1ª edição

Relatório apresentado para a matéria de Compiladores (código GCC130) contendo discussões sobre as decisões de projeto e dificuldades encontradas, além da apresentação de dois diagramas de transição (DFAs) referentes à classes de *tokens* e a inclusão de um arquivo de teste e sua saída.

LAVRAS – MG

2026

SUMÁRIO

1 Introdução 2

2 Decisões de Projeto 3

2.1 Política da Tabela de Símbolos 3

2.2 Estratégia de Tokenização 3

2.3 Regras Léxicas e Identificadores 4

2.4 Implementação Técnica 4

3 Dificuldades Encontradas 5

4 Diagramas de Transição (DFAs) 6

4.1 DFA para Identificadores 6

4.2 DFA para Números Inteiros 6

5 Arquivo de Teste e Saída 7

5.1 Código de Entrada (test-4.txt) 7

5.2 Saída Gerada pelo Analisador 7

5.3 Tabela de Símbolos Resultante 9

6 Conclusão 10

1 INTRODUÇÃO

Este relatório descreve o desenvolvimento da primeira etapa do compilador para a disciplina GCC130. O objetivo do trabalho prático é desenvolver um compilador para uma mini linguagem didática. Nesta fase inicial, o foco recai sobre o Analisador Léxico, implementado com a ferramenta Flex. O scanner é responsável por identificar elementos fundamentais como declarações de tipos (int, float), identificadores, palavras-chave (if, else, while, print, read), operadores relacionais, aritméticos e lógicos, além de símbolos de pontuação e o tratamento de comentários.

2 DECISÕES DE PROJETO

As decisões técnicas tomadas pela equipe visaram a eficiência da análise e a facilidade de integração com o analisador sintático nas etapas posteriores.

2.1 Política da Tabela de Símbolos

A tabela de símbolos armazena apenas identificadores válidos reconhecidos pela regra de identificador. Essa escolha reduz o volume de dados persistidos no léxico e mantém a tabela focada em nomes do programa (variáveis, funções, etc.), que serão consultados nas etapas semânticas. Constantes numéricas não são inseridas na tabela; elas são retornadas como tokens numéricos com atributo semântico próprio.

2.2 Estratégia de Tokenização

Foi adotada uma abordagem híbrida de classificação de tokens:

- **Grupos unificados com yylval:** Conjuntos como TYPE (int, float), RELOP (le, ge, ne, eq, lt, gt), LOGIC_OP (and, or, neg) e NUMBER (int, float) foram mapeados para uma única classe de token. A distinção específica é mantida no atributo semântico, o que simplifica o parser e reduz o número de regras na gramática.
- **Grupos mantidos separados:** Operadores aritméticos e símbolos de pontuação permaneceram como tokens distintos. Nesses casos, a análise sintática se beneficia da distinção explícita para tratar precedência, associatividade e o papel estrutural de cada símbolo.

2.3 Regras Léxicas e Identificadores

As regras de inteiro e ponto flutuante não incluem sinal opcional no léxico; assim, os símbolos $+$ e $-$ são sempre tratados como tokens próprios de operador. Isso evita ambiguidades léxicas e delega ao parser a decisão entre uso binário ou unário. Além disso, foi adicionada uma regra específica para detectar identificadores inválidos (que iniciam com dígito, começam com underline ou terminam com underline), permitindo mensagens informativas que diferenciam esses erros de sequências inválidas genéricas.

2.4 Implementação Técnica

A tabela de símbolos foi implementada utilizando uma estrutura hash de tamanho fixo primo (TABLE_SIZE = 211) com tratamento de colisões por encadeamento. A estrutura Symbol armazena o lexema, classe do token, linha, coluna e ponteiro para o próximo nó. Funções auxiliares como resetColumnCounter e incrementColumnCounter foram definidas para centralizar a atualização de colunas e garantir que o código das regras léxicas permaneça limpo.

3 DIFICULDADES ENCONTRADAS

Durante o desenvolvimento, a equipe enfrentou os seguintes desafios:

- **Política de Maior Casamento do Flex:** A detecção de erros léxicos de maior extensão, como comentários multilinha sem fechamento, foi dificultada pela tendência do Flex de consumir o trecho mais longo possível. Adotou-se um fallback de caractere único como erro léxico genérico para garantir o progresso da varredura e evitar que erros amplos mascarem a tokenização de trechos seguintes.

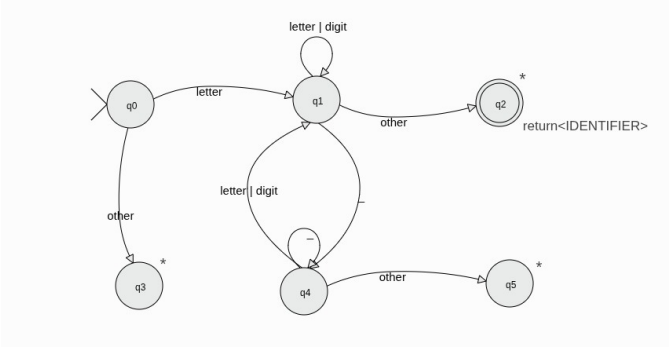
- **Macros em Classes de Caracteres:** Houve dificuldade técnica ao assumir que o Flex expandiria definições regulares dentro de colchetes, o que não ocorre. Esse detalhe gerou erros de sintaxe regex difíceis de rastrear, reforçando a necessidade de uma estratégia de depuração incremental.

4 DIAGRAMAS DE TRANSIÇÃO (DFAS)

Apresentam-se os diagramas de transição para as classes de tokens exigidas.

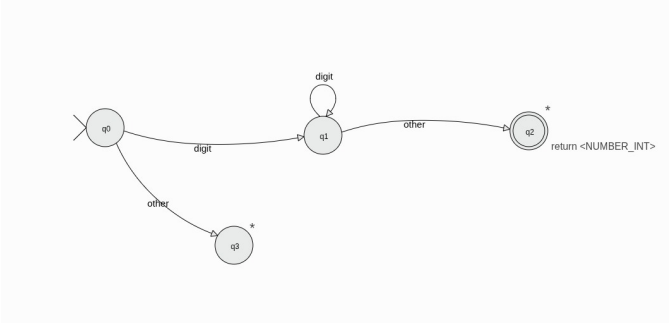
4.1 DFA para Identificadores

Figura 4.1 – Autômato finito para identificadores válidos.



4.2 DFA para Números Inteiros

Figura 4.2 – Autômato finito para a classe Integer.



5 ARQUIVO DE TESTE E SAÍDA

A validação do analisador foi realizada com o arquivo `test-4.txt`.

5.1 Código de Entrada (test-4.txt)

```
// Casos com simbolos invalidos
@@
###
$
??
!~
// Mistura de validos e invalidos
abc@@def
12#34
_naoIniciaComLetra
valor = 2 + 10;
valor2 = 2-10;
// Possivel comentario nao fechado
/* comentario sem fechamento
```

5.2 Saída Gerada pelo Analisador

A execução demonstra a detecção de erros e identificação de tokens com linha e coluna:

```
2(1): @@ (LEXICAL_ERROR)
3(1): ### (LEXICAL_ERROR)
4(1): $ (LEXICAL_ERROR)
5(1): ?? (LEXICAL_ERROR)
6(1): !~ (LEXICAL_ERROR)
9(1): abc (IDENTIFIER)
```

```
9(4): @@ (LEXICAL_ERROR)
9(6): def (IDENTIFIER)
10(1): 12 (NUMBER (NUMBER_INT))
10(3): # (LEXICAL_ERROR)
10(4): 34 (NUMBER (NUMBER_INT))
11(1): _naoIniciaComLetra (LEXICAL_ERROR)
12(1): valor (IDENTIFIER)
12(7): = (ASSIGN_OP)
12(9): 2 (NUMBER (NUMBER_INT))
12(11): + (ARITH_OP_ADD)
12(13): 10 (NUMBER (NUMBER_INT))
12(15): ; (PUNCT_SEMICOLON)
13(1): valor2 (IDENTIFIER)
13(8): = (ASSIGN_OP)
13(10): 2 (NUMBER (NUMBER_INT))
13(11): - (ARITH_OP_SUB)
13(12): 10 (NUMBER (NUMBER_INT))
13(14): ; (PUNCT_SEMICOLON)
15(1): /* (LEXICAL_ERROR)
15(4): comentario (IDENTIFIER)
15(15): sem (IDENTIFIER)
15(19): fechamento (IDENTIFIER)
```

5.3 Tabela de Símbolos Resultante

Após a execução sobre o arquivo de teste, a tabela de símbolos continha os seguintes registros:

Tabela 5.1 – Conteúdo da tabela de símbolos após o teste.

Token	**Lexema**
IDENTIFIER	comentario
IDENTIFIER	fechamento
IDENTIFIER	valor2
IDENTIFIER	valor
IDENTIFIER	def
IDENTIFIER	abc
IDENTIFIER	sem

6 CONCLUSÃO

A Etapa 1 foi concluída com sucesso, resultando em um analisador léxico robusto que cumpre os requisitos de identificação de tokens, gerenciamento de símbolos via hash e reporte de erros detalhado. O projeto segue para a análise sintática em Bison.